

Using iPhone Mic on Windows

<https://gemini.google.com/app/2b30751c71de0ad6>

User prompt: Is it possible to write a custom app to essentially use the mic of an iPhone to use it in a Windows device as mic? basically wommic but that uses tcp instead of udp or connecting via lightning cable (usb-c to lightning)

Response: Yes, it is entirely possible to build this. In fact, your specific requirement—using TCP over a physical Lightning or USB-C cable—solves one of the biggest problems with wireless audio streaming: packet loss and latency.

While UDP is the standard for real-time audio over Wi-Fi (because dropping an audio packet is better than pausing the stream to wait for it), **TCP over a wired USB connection is practically flawless** since packet loss is virtually zero.

To build this, you will need to create two pieces of software and use a specific tunneling trick to get them to talk over the cable. Here is exactly how the architecture works.

1. The iOS App (The Sender)

Your iPhone app's job is straightforward: capture microphone input and blast it out through a TCP socket.

- **Audio Capture:** You will use Apple's `AVFoundation` framework, specifically `AVAudioEngine`. You can attach a "tap" to the input node (`inputNode.installTap`), which gives you a continuous stream of PCM audio buffers in real-time.
- **Networking:** Use Apple's `Network` framework (`NWListener` or `NWConnection`) to open a TCP port (e.g., port 12345) and host a local server on the iPhone. When a client connects, the app starts writing the PCM audio buffers directly to the TCP stream.
- **Background Execution:** To ensure the mic doesn't turn off when you lock the screen, you must add the `UIBackgroundModes` key with the `audio` value in your app's `Info.plist`.

2. The USB Cable Bridge (usbmuxd)

This is the "magic" step. iOS does not allow standard apps to act as raw USB gadgets (you can't just plug in an iPhone and tell Windows "I am a USB microphone"). However, you can route standard TCP traffic over the USB cable using Apple's multiplexing daemon, **usbmuxd**.

This is the exact same protocol iTunes and Xcode use to communicate with iPhones over USB.

- On the Windows side, you can use a port-forwarding tool like `iproxy` (part of the open-source [libimobiledevice](#) library).

- Running `iproxy 12345 12345` on Windows will bind a local port on your PC to the TCP port your iOS app is hosting.
- The result: Your Windows app connects to `localhost:12345`, and `usbmuxd` physically routes that TCP connection through the Lightning/USB-C cable directly into your iOS app.

3. The Windows App (The Receiver)

The Windows application will connect to the local TCP port, receive the raw audio data, and inject it into the system as a microphone.

- **Receiving Data:** You can write this in C#, C++, or Python. The app simply connects to `12345` via standard TCP sockets and reads the incoming byte stream.
- **The Virtual Microphone Problem:** This is the hardest part of the project. Windows will not let a user-level application pretend to be a hardware microphone without a custom kernel-mode audio driver. Writing and code-signing a Windows kernel driver (Sysvad) is expensive and notoriously difficult.
- **The Solution:** Do what almost every indie audio app does—use a **Virtual Audio Cable**. You can install a free driver like [VB-Cable](#). It creates a virtual "Playback" device (Speaker) and a virtual "Recording" device (Microphone) that are hardwired together.
- Your Windows app will use an audio library (like NAudio for C# or PortAudio) to **play** the received iOS audio directly into the "VB-Cable Input" speaker. Windows will then automatically route that sound to the "VB-Cable Output" microphone, which you can select in Discord, Zoom, or OBS.

Why this approach is highly effective

1. **Zero-Latency Encoding:** Because you are using a USB cable, bandwidth is incredibly high (480 Mbps for Lightning/USB 2.0, up to 10 Gbps for USB-C). You don't even need to waste CPU cycles compressing the audio into Opus or AAC. You can stream raw, uncompressed 48kHz PCM audio directly over the TCP socket for near-zero latency.
 2. **Reliability:** Wi-Fi apps like WO Mic often suffer from jitter or interference. A wired TCP connection via `usbmuxd` will never drop or stutter.
-